



Revision Notes for DSA Lab Session on Hashing

Session Overview

This session delves into crucial concepts of hashing within data structures and algorithms. Key problems were addressed, demonstrating how hashing can be leveraged to solve a variety of problems efficiently.

Concepts and Problems Discussed

Longest Substring Without Repeating Characters

Problem Statement:

Given a string s , find the length of the longest substring without repeating characters.

Example:

- Input: "abcabcbb"

Output: 3

Explanation: The longest substring without repeating characters is "abc", with a length of 3.

Approach:

- Utilize a sliding window technique with a hash set to track characters within the current window.
- Increase the window size by moving the end pointer and check for the uniqueness of characters using the hash set.
- If a duplicate character is found, adjust the start pointer until the substring is unique again.

Complexity:



- **Space Complexity:** $O(\min(n, m))$, where n is the length of the string, and m is the size of the character set `【4:13+source】` .

First Non-Repeating Element in an Array

Problem Statement:

Given an array of integers, find the first non-repeating element.

Example:

- Input: [2, 3, 3, 2, 4] Output: 4

Brute Force Approach:

- Traverse every element and check for its repetition through another traversal, leading to an $O(n^2)$ complexity.

Optimized Approach Using Hash Map:

- First, create a frequency hash map of the elements.
- Make a second pass to check the first element with a frequency of one.

Complexity:

- **Time Complexity:** $O(n)$ for both creating the hash map and checking the element frequencies.
- **Space Complexity:** $O(n)$ for storing elements in the hash map `【4:6+source】` .

Subarray with Sum Equal to Zero

Problem Statement:

Determine if there exists a subarray within a given array that sums to zero.

Example:

- Input: [1, 2, -3, 3] Output: True (subarray [1, 2, -3])



- Calculate a running prefix sum while traversing the array.
- Use a hash set to track these prefix sums. If a prefix sum repeats or is zero, a subarray with zero sum exists.

Complexity:

- **Time Complexity:** $O(n)$ to check through the array.
- **Space Complexity:** $O(n)$ for storing prefix sums in the hash set
【4:14+source】.

Subarray with Sum Equal to a Given Value (K)

Problem Statement:

Find if there's any subarray with a sum that equals a given value K .

Approach:

- Similar to the zero-sum subarray problem, compute the prefix sums while storing them in a hash map.
- For each prefix sum P , check if $P - K$ exists in the hash map to determine the subarray with sum K exists.

Complexity:

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(n)$ 【4:10+transcript.txt】.

Key Takeaways

- **Hash Maps and Sets:** They are powerful tools for solving problems involving frequency counts and quick lookups, efficiently solving what would otherwise be complex time-consuming problems.
- **Sliding Window Technique:** An efficient approach to find substrings or subarrays under certain conditions, such as uniqueness or sum constraints.
- **Dynamic Problem Solving:** Convert brute force solutions into more optimal algorithms through careful observation and application of known data structures and methodologies.



Scaler Companion

beta

your problem-solving skills especially in competitive programming
and technical interviews.